

Certified support cuts for shared nonlinear expressions and quadratic blocks

October 2026

Abstract

Several nonlinear expressions can share variables and affine constraints that separate scalar relaxations do not fully use. This report develops an implemented support-cut interface for these dependencies. Candidate directions may be chosen numerically; a separate calculation checks the final coefficients on their complete domain before a cut is submitted. Exact rational Bernstein bounds cover supported polynomial blocks, and outward interval evaluation covers a stated elementary-function grammar. We give exact rational support algorithms for quadratic vector graphs over bounded two-dimensional polyhedra and for quadratic stars with affine center-leaf constraints. We also characterize a limitation of intersecting overlapping pair hulls and give a quantitative separating example. Automatic discovery, bounded separation, cached work, and an optional native sampling kernel connect the construction to SCIP while retaining native nonlinear feasibility enforcement. A fixed short-run experiment and certificate replay evaluate supported mechanisms and their costs. The report separates cut validity from numerical solver bounds and feasible residual checks. Of 24 held-out models, 20 pass the common research importer. Native baseline and the reformulation control each solve 19 of those 20 within the short budget; both cut modes solve 18. Automatic selection reduces separation work, but these results do not support default activation. The hull, Bernstein, and sparse quadratic ingredients have substantial prior literature; this is an integrated research implementation with explicit mathematical and computational limits, not a claim of general solver acceleration or publication priority.

Contents

1	The capability and the evidence standard	2
2	Joint support cuts	3
3	Finite certificates on the whole domain	4
3.1	Exact Bernstein bounds for rational polynomials	4
3.2	Supported elementary expressions and explicit failure	5
4	A checked reason to skip separation	5
5	Exact support for coupled quadratic pairs	6
6	A larger exact class: quadratic stars with affine rows	8
7	What overlapping pair hulls miss	9
7.1	The stronger interface that makes tree gluing valid	11

8	Implementation and model binding	11
8.1	What the certificate actually checks	11
8.2	Discovery and native enforcement	13
8.3	Automatic activation and bounded repeated work	13
8.4	The native sampling kernel	14
9	Computational evidence and verification	14
9.1	The frozen comparison	14
9.2	Primary outcome: fewer held-out solves with cuts	15
9.3	Root strength, work selection, and ablations	16
9.4	Replay coverage and numerical residual checks	17
9.5	The exact overlap mechanism	17
9.6	Native sampling measurements	17
9.7	Targeted checks and actual-file review	18
10	Relationship to established results	19
11	What is complete and what remains outside the result	20

1 The capability and the evidence standard

The task is to strengthen relaxations automatically when several nonlinear expressions move together. The existing repository already contains scalar composite envelopes, simultaneous univariate-curve cuts, and concave row hulls. Their recorded results motivate this continuation, but they do not establish a generally beneficial solver component. In particular, the historical composite-univariate campaign solved 51 of 107 instances within 120 seconds with its Python handler, versus 54 with native SCIP. Its curvature classification was validated with ball arithmetic, while function values in final cuts used a floating-point safety shift. The historical curve separator also documented a numerical-library accuracy assumption. These are historical results, not reruns or certified outcomes of this continuation. The original records are [results/composite-univariate-envelopes.md](#) and [research-20260922/curve-hulls/report.md](#).

This report closes a narrower and testable development program: a checked cut interface on explicit supported expression classes; automatic block discovery and bounded repeated work; exact support for coupled quadratic blocks and a larger structured extension; comparative solver evidence; and a traceable comparison with primary literature. A supported cut is checked against a supplied expression graph and domain. A completed SCIP solve is not thereby converted into a replayable global optimality proof.

The distinction matters at three levels. First, a mathematical support theorem proves validity for every feasible graph point. Second, a replay record checks a concrete cut for a concrete model representation. Third, SCIP reports numerical primal and dual information after its own presolve, LP computations, transformations, and search. Only the first two levels are covered by the new cut-certificate claim. Numerical primal residuals and comparisons with known objectives are useful independent checks, but are not exact feasibility or dual-bound certificates.

The companion [COVERAGE.md](#) maps claims to proofs, code, saved outputs, and limitations. The frozen experiment protocol is [experiments/protocol.md](#). Internal research-agent reviews are recorded separately from the construction; they are not journal peer review or formal verification.

2 Joint support cuts

Let $D \subset \mathbb{R}^n$ be nonempty and compact, and let $F = (f_1, \dots, f_k) : D \rightarrow \mathbb{R}^k$ be continuous. Write

$$G_D = \{(x, F(x)) : x \in D\}, \quad H_D = \text{conv } G_D.$$

The lifted variables w_j represent the *same* functions f_j of the *same* variables x . For a direction $d = (a, b) \in \mathbb{R}^{n+k}$ define

$$h_D(a, b) = \min_{x \in D} \{a^\top x + b^\top F(x)\}. \quad (1)$$

We use lower support values, so every cut is written with a lower bound. If integer variables occur in D , support over a continuous superset remains valid for the mixed-integer graph, but need not describe its hull.

Proposition 2.1 (Support and complete hull description). *If $\underline{h} \leq h_D(a, b)$, then*

$$a^\top x + b^\top w \geq \underline{h} \quad (2)$$

is valid for H_D . Moreover,

$$H_D = \bigcap_{(a,b) \in \mathbb{R}^{n+k}} \{(x, w) : a^\top x + b^\top w \geq h_D(a, b)\}.$$

Proof. The defining minimum makes the inequality valid at each point of G_D ; linearity preserves it under convex combinations. The graph is compact by continuity, and its convex hull is compact in finite dimension. A point outside that hull is strictly separated by a linear functional. Orient that functional as a lower support inequality to obtain a member of the displayed intersection that excludes the point. $\square$

This is classical convex hull duality; simultaneous graph separation from linear combinations is developed explicitly in [15, 21]. The computational problem is to evaluate or bound (1), and to find a useful direction without spending more than its benefit. A numerical direction generator is permitted to be inaccurate: its output only chooses (a, b) . Soundness depends on a bound for that *final* direction, not on the correctness or optimality of the direction search. The same observation allows random samples, LP direction search, and a native sampling kernel outside the cut-validity calculation.

For $n = 1$ and $k = 1$, optimizing over the coefficient of x recovers the biconjugate description of a scalar convex envelope. For $k > 1$, joint directions retain dependencies between outputs that intersected scalar envelopes can omit. A finite collection of valid joint cuts generally gives an outer approximation; absence of a violated generated cut does not establish membership in H_D .

Proposition 2.2 (Final floating-point row). *Suppose the exact values of the coefficients submitted to a numerical solver are $\hat{a} \in \mathbb{Q}^n$ and $\hat{b} \in \mathbb{Q}^k$ (in particular, finite binary floating-point numbers interpreted exactly). If a checker establishes*

$$L \leq \min_{x \in D} \{\hat{a}^\top x + \hat{b}^\top F(x)\}$$

and the submitted right-hand side $\hat{L}$ satisfies $\hat{L} \leq L$ exactly, the submitted mathematical row is valid on H_D .

Proof. Apply Proposition 2.1 to the submitted coefficients and then weaken the right-hand side from L to $\hat{L}$. $\square$

Rounding an already proved rational row without checking its resulting coefficients would not satisfy this contract. One alternative, when a trusted box $\ell \leq z \leq u$ is available for all lifted coordinates, is to replace the exact row $c^\top z \geq L$ by

$$\hat{c}^\top z \geq L + \sum_i \min\{(\hat{c}_i - c_i)\ell_i, (\hat{c}_i - c_i)u_i\},$$

and round the right-hand side downward. The formula follows by bounding each coordinate of $(\hat{c} - c)^\top z$. Our principal contract is the direct check for the final direction in Proposition 2.2; it avoids needing auxiliary-variable bounds for this correction.

3 Finite certificates on the whole domain

3.1 Exact Bernstein bounds for rational polynomials

Consider a polynomial p on a rational box $B = \prod_{i=1}^n [\ell_i, u_i]$. Substitute $x_i = \ell_i + (u_i - \ell_i)t_i$ and eliminate fixed coordinates. In the remaining coordinates, write

$$p(x(t)) = \sum_{\beta \leq d} a_\beta t^\beta = \sum_{\alpha \leq d} b_\alpha B_\alpha^d(t), \quad B_\alpha^d(t) = \prod_i \binom{d_i}{\alpha_i} t_i^{\alpha_i} (1 - t_i)^{d_i - \alpha_i}.$$

Here d is a componentwise degree bound and all coefficients are exact rationals. The conversion is

$$b_\alpha = \sum_{\beta \leq \alpha} a_\beta \prod_i \frac{\binom{\alpha_i}{\beta_i}}{\binom{d_i}{\beta_i}}. \quad (3)$$

Lemma 3.1 (Bernstein enclosure). *On B , $\min_\alpha b_\alpha \leq p \leq \max_\alpha b_\alpha$.*

Proof. For $0 \leq t_i \leq 1$, every $B_\alpha^d(t)$ is nonnegative and $\sum_\alpha B_\alpha^d(t) = 1$, by the binomial theorem in each coordinate. Thus $p(x(t))$ is a convex combination of the coefficients. Identity (3) follows by expressing each monomial t^β in the tensor-product Bernstein basis; the univariate identity has coefficients $\binom{\alpha}{\beta} / \binom{d}{\beta}$. $\square$

The enclosure is classical [6, 5]. The implementation contribution is to bind the exact polynomial, its original box, and the checked final row to the recorded partition and lower bounds. Adaptive subdivision can improve a weak enclosure without changing the validity argument.

Proposition 3.2 (Partition certificate). *Let $B_1, \dots, B_N$ be finitely many closed boxes whose union is B , and suppose $L_j \leq \inf_{B_j} p$ is verified for each j . Then $L = \min_j L_j \leq \inf_B p$. The same conclusion holds for a finite interval partition and any expression with a verified interval enclosure on every cell.*

Proof. Every $x \in B$ belongs to a checked cell B_j , where $p(x) \geq L_j \geq \min_i L_i$. $\square$

The coverage premise is essential. Checking all supplied cells while accepting an omitted end interval would not prove the claimed bound. The certificate checker must start from the trusted original domain and verify the entire subdivision, including endpoints. A cache key also needs the expression, direction, and domain: reusing a valid bound from a smaller domain on a larger domain is unsound.

3.2 Supported elementary expressions and explicit failure

For a supported elementary expression, outward ball or interval arithmetic supplies an enclosure for the support objective on each interval. Rational constants and final binary coefficients are embedded exactly. Expression evaluation must retain domain requirements: a logarithm needs a positive argument; division needs a denominator excluding zero; powers require their stated real domain. Algebraic simplification is not permission to remove the original expression's domain restrictions.

If a pole, invalid domain, nonfinite enclosure, unsupported operation, or resource limit prevents a complete bound, the procedure returns that status. An exhausted direction search means only that no checked useful cut was obtained within its budget. It does not prove that a point lies in a hull, that the original constraints hold, or that the domain is empty. Even a successfully checked cut need not be violated enough to justify insertion into the LP.

The elementary support calculation can also use a curvature correction. If the scalar direction p is twice continuously differentiable on $[a, b]$ and a verified bound gives $p'' \leq M$, then

$$\min_{[a,b]} p \geq \min\{p(a), p(b)\} - \max\{0, M\} (b - a)^2 / 8. \quad (4)$$

Indeed, $p(t) - Mt^2/2$ is concave, so its chord inequality rearranges to $p(t) \geq \text{chord}_p(t) - M(t - a)(b - t)/2$. Use $(t - a)(b - t) \leq (b - a)^2/4$ when $M \geq 0$; for $M < 0$, the chord itself is a lower bound. Verified lower endpoint values can replace $p(a)$ and $p(b)$. The implementation first checks all feature domains, including zero-weight features, then uses the maximum of this bound and available natural enclosures. An unavailable derivative bound or singular derivative disables this optional strengthening; it does not license removing a domain endpoint.

The supported grammar, actual certificate schema, and solver binding are stated with their implementation in Section 8. Neither Python's rational arithmetic nor the interval library is formally verified here. Replay reduces the trusted computational path and catches mismatched artifacts; it is not a proof of the underlying software stack.

4 A checked reason to skip separation

A support oracle should not run merely because a block exists. One inexpensive certificate can rule out cuts with sufficiently large normalized violation at the current query. This is a local statement about a chosen normalization and tolerance, not a prediction of total solver time.

Let $g(x) = (x, F(x)) \in \mathbb{R}^r$ denote the feature vector used by a block, let $\bar{z} \in \mathbb{R}^r$ be the current lifted query, and choose positive coordinate scales $s_1, \dots, s_r$. Suppose $x^1, \dots, x^N \in D$ are verified parameter samples and $\lambda_i \geq 0$, $\sum_i \lambda_i = 1$. Put $z^* = \sum_i \lambda_i g(x^i) \in H_D$. With exact graph values, define

$$\rho_1 = \sum_j \frac{|\bar{z}_j - z_j^*|}{s_j}, \quad \rho_\infty = \max_j \frac{|\bar{z}_j - z_j^*|}{s_j}.$$

The implementation can instead have verified enclosures $g_j(x^i) \in [\ell_{ij}, u_{ij}]$. Define $L_j = \sum_i \lambda_i \ell_{ij}$, $U_j = \sum_i \lambda_i u_{ij}$, and

$$r_j = \frac{\max\{|\bar{z}_j - L_j|, |\bar{z}_j - U_j|\}}{s_j}, \quad R_1 = \sum_j r_j, \quad R_\infty = \max_j r_j. \quad (5)$$

All quantities in this expression are rational in the certificate.

Theorem 4.1 (Normalized violation screening). *For every valid lower support inequality $c^\top z \geq \beta$ on H_D ,*

$$\beta - c^\top \bar{z} \leq R_1 \quad \text{if } \max_j s_j |c_j| \leq 1,$$

and

$$\beta - c^\top \bar{z} \leq R_\infty \quad \text{if } \sum_j s_j |c_j| \leq 1.$$

Consequently, $R_1 \leq \tau$ rules out support cuts with violation strictly larger than τ under the first normalization; the corresponding statement holds for R_∞ under the second.

Proof. The actual convex combination z^* belongs to H_D and satisfies $L_j \leq z_j^* \leq U_j$. Validity gives

$$\beta - c^\top \bar{z} \leq c^\top (z^* - \bar{z}) \leq \sum_j (s_j |c_j|) \frac{|z_j^* - \bar{z}_j|}{s_j}.$$

Bound the last expression by R_1 when each first factor is at most one, or by R_∞ when their sum is at most one. These are the finite-dimensional Hölder inequalities with an interval upper bound for each residual. $\square$

Proposed convex weights may come from an ordinary numerical LP. To obtain a valid witness, interpret its finite coefficients exactly, replace negative weights by zero, and divide by their positive exact sum. This preserves a convex combination regardless of the accuracy of the proposal. An all-nonpositive vector supplies no witness. Each sample must satisfy the exact box and affine rows, and its original graph value must be enclosed by the trusted evaluator. A sample feasible only for a relaxation of the *block domain being certified* is not sufficient for this screening theorem. If the block deliberately uses a larger domain than the original model, the conclusion concerns cuts for that larger-domain hull.

The screen is safe to omit: a large residual, failed LP, rejected sample, or failed enclosure causes ordinary bounded separation to continue. For a positive tolerance, a successful screen does not prove exact hull membership or original nonlinear feasibility. At exactly zero residual, it proves membership of the query in the chosen block hull, which still does not imply membership of the query in the original graph or in the full model’s simultaneous hull.

The integration’s direction LP bounds each scaled direction coordinate individually, so the applicable certificate is R_1 , not R_∞ . The two variants are implemented explicitly in [solver/screening.py](#). Sample enclosures are cached against a fixed evaluator and domain. Serialized replay re-evaluates the original graph and verifies the proposed mixture, query, normalization, and threshold; serialized enclosures alone are not trusted. This theorem is standard convex-combination geometry. Its role is to give a concrete, checkable skip condition, separate from the heuristic decision to spend time looking for one.

5 Exact support for coupled quadratic pairs

Let $P \subset \mathbb{R}^2$ be a bounded polyhedron described by rational bounds and rational affine inequalities. The full quadratic feature graph uses

$$v(x, y) = (1, x, y, x^2, xy, y^2).$$

Any direction for a vector of quadratic expressions gives a quadratic objective

$$q(x, y) = c_0 + c_x x + c_y y + c_{xx} x^2 + c_{xy} xy + c_{yy} y^2. \tag{6}$$

All coefficients are rational, including exactly interpreted finite binary floating-point inputs.

Theorem 5.1 (Finite rational support candidates). *If P is nonempty, the minimum of q is attained at one of the following rational points:*

1. *a vertex of P ;*
2. *a stationary point in the relative interior of a polygon edge, or of P itself when P is a segment, on which the restriction of q has strictly positive quadratic coefficient;*
3. *the stationary point of q in P , if its Hessian is positive definite and P is two-dimensional.*

Enumerating this list and evaluating it exactly gives an attained exact support value. Empty, segment, and singleton domains are included.

Proof. The compact nonempty domain has a minimizer. If it lies on the boundary of a two-dimensional polygon, or if the domain has dimension at most one, restrict q to the relevant closed segment. Its minimum is at an endpoint unless its quadratic coefficient is positive and its stationary point is interior; those are the listed candidates.

If a minimizer is interior in two dimensions, its gradient vanishes and its constant Hessian H is positive semidefinite. If H is positive definite, the unique stationary point is the solution of a nonsingular rational linear system. If H is singular, choose a nonzero vector $d \in \ker H$. Along the line through the minimizer z , the quadratic identity and $\nabla q(z) = 0$ give $q(z + td) = q(z)$. The line reaches the boundary of the bounded polygon, so a boundary minimizer also exists. The zero Hessian case is included. Thus no extra singular interior candidate is required. Every vertex is obtained from rational active rows, and edge stationary parameters are rational ratios. Exact evaluation therefore gives a rational minimum and rational minimizer. $\square$

With m affine rows including bounds, the implementation intersects all pairs of nonparallel boundary lines and checks each intersection against every row. A planar hull orders the retained vertices. This costs $O(m^3)$ exact rational operations for the straightforward enumeration; the edge and interior candidates add lower-order work. This is an arithmetic-operation bound, not a floating-point runtime guarantee. Rational arithmetic must account for numerator and denominator growth.

The certificate stores the original box, every affine row, every coefficient, polygon vertices, all candidates, their exact values, and the minimizer. Replay reconstructs the list from trusted inputs. The producer and checker share geometric routines; this is reproducible exact arithmetic with independent direction search, not an independently implemented formal checker.

Using the row domain before convexifying can strengthen even an exact box graph hull. On $x, y \geq 0, x + y \leq 1$,

$$x^2 + 2xy + y^2 \leq x + y, \quad xy \leq 1/4.$$

The first inequality is $s^2 \leq s$ for $s = x + y \in [0, 1]$, and the second follows from $4xy \leq (x + y)^2$. In contrast, half mass at each of $(0, 0)$ and $(1, 1)$ is a valid box graph-hull point whose means satisfy the row, but whose second-moment coordinates give $x^2 + 2xy + y^2 = 2 > x + y = 1$ and $xy = 1/2$. Thus intersecting the box hull with the row is strictly weaker than convexifying the row domain.

This oracle solves support minimization for every vector of quadratic expressions over the same pair domain. It does not promise to discover every violated direction in bounded time. More general bivariate polynomials use the Bernstein route when supported, with a potentially large subdivision cost.

6 A larger exact class: quadratic stars with affine rows

The obstruction in Section 7 motivates support directions that span adjacent pairs. A tractable extension is a center variable y with leaves $x_1, \dots, x_k$. The objective is

$$q(y, x) = a_0 + a_1 y + a_2 y^2 + \sum_{i=1}^k \{d_i x_i^2 + (e_i y + f_i) x_i\}. \quad (7)$$

All objective coefficients, affine-row coefficients, right-hand sides, and finite variable bounds are rational; finite binary floating-point inputs mean their exact rational values. Each supplied affine row contains only y and at most one leaf. Rows containing only y are allowed; rows involving two different leaves are outside this class. Every quadratic coefficient can have either sign. The same structure must hold for the scalar support objective obtained from a proposed direction.

For fixed y , each leaf is constrained to an interval

$$L_i(y) \leq x_i \leq U_i(y), \quad L_i(y) = \max_{r \in \mathcal{L}_i} (\alpha_{ir} + \beta_{ir} y), \quad U_i(y) = \min_{r \in \mathcal{U}_i} (\gamma_{ir} + \delta_{ir} y). \quad (8)$$

The original bounds provide at least one function in each envelope. Projecting each bounded center-leaf polygon onto the center gives a closed rational interval. Their intersection, together with the center bounds and center-only rows, is exactly the feasible center interval I : conditional leaf feasibility is independent across leaves. An empty intersection proves this support domain empty.

Theorem 6.1 (Exact rational star support). *For the nonempty domain just described, the minimum of (7) is rational and has a rational minimizer. It is obtained by a finite rational partition of I on which each leaf has an affine minimizer and the conditional optimum is quadratic in y . Both the partition and the attained minimum can be reconstructed exactly from the supplied rows, bounds, and coefficients.*

Proof. Partition I at all intersections of affine functions in the envelopes of each leaf. On every open resulting interval, the active functions L_i and U_i are fixed and affine.

If $d_i > 0$, the unconstrained leaf minimizer is the affine function $v_i(y) = -(e_i y + f_i)/(2d_i)$. The constrained minimizer is $\max\{L_i(y), \min\{v_i(y), U_i(y)\}\}$. Further partition at intersections of v_i with its two active bounds. Each remaining interval has one affine minimizing rule.

If $d_i \leq 0$, a minimizing leaf value is an endpoint. The difference between the upper and lower endpoint objectives factors as

$$q_i(y, U_i(y)) - q_i(y, L_i(y)) = (U_i(y) - L_i(y))\{d_i(U_i(y) + L_i(y)) + e_i y + f_i\}.$$

The first factor is nonnegative on the feasible center interval. The second is affine on the current interval, so at most one additional rational root determines the preferred endpoint; an identically zero factor permits either endpoint. This also gives an affine minimizing rule after refinement.

Take the union of all leaf breakpoints. Substituting the selected affine rules into (7) gives a rational quadratic on every closed piece. The rule remains feasible and minimizing at its endpoints: bound switches coincide there, and endpoint-choice switches have equal values there. Minimize each quadratic at its endpoints and, for a positive square coefficient, its feasible stationary point. These candidates are rational and exhaustive. The selected center and leaf rules supply an attained rational minimizer. A singleton center interval uses the same conditional scalar minimization at that point. $\square$

If leaf i has m_i envelope lines, their pair intersections produce at most $O(m_i^2)$ center breakpoints; a constant number of further switches per envelope interval gives the same order. The union of breakpoints has polynomial size. The present implementation deliberately uses straightforward pair enumeration and scanning. Its arithmetic cost is polynomial in the supplied row counts and number of leaves; no claim of an optimal complexity bound is needed for this small-block oracle. Unlike general elimination on a deeper graph, this construction does not repeatedly nest piecewise messages along a tree.

The implementation [theory/quadratic_star.py](#) records the complete center partition, affine leaf rules, quadratic value functions, and attained minima. Unsupported mixed terms or rows raise an explicit error instead of being dropped. Replay rebuilds the same exact object from trusted input. As with the polygon oracle, this is exact support for a supplied direction; completeness of the integration's direction search is a separate question.

Corollary 6.2 (Sharp gain for specified pair directions). *Let compact rational center-leaf polygons P_i assemble into a nonempty star, and fix rational quadratic pair objectives $q_i(y, x_i)$. Define*

$$\beta_i = \min_{P_i} q_i, \quad \beta_* = \min_{(y, x_i) \in P_i \ \forall i} \sum_i q_i(y, x_i), \quad \Delta = \beta_* - \sum_i \beta_i.$$

Then $\Delta \geq 0$ is the largest possible constant improvement in the sum of these fixed pair inequalities. It is exactly computable by the polygon and star support algorithms. Moreover, $\Delta > 0$ if and only if the center projections of the pair minimizing sets have no common point.

Proof. Every pair objective is at least its individual minimum, proving $\Delta \geq 0$. Equality holds precisely when one feasible assembled point minimizes every pair objective. Such a point exists precisely when their minimizing center sets intersect, since leaves can then be chosen independently. If no such point exists, compactness makes the attained assembled minimum strictly larger. The attained value β_* is the largest valid constant in this summed direction. $\square$

This gives a precise diagnostic for a proposed merger of directions. It does not choose the normals or predict whether computing the gain will improve total runtime. In the example below, both pair minima are zero and the exact gain is $1/128$.

The box-only case is a specialization of Del Pia and Khajavirad's forest-QP algorithm, which gives an $O(n^2)$ arithmetic bound for general forests [19]. Piecewise-affine responses of strictly convex parametric quadratic programs are also classical [3]. The additional domain handled explicitly here consists of center-leaf affine rows with all signs of scalar leaf curvature. That scope distinction and its exact implementation are documented; a first-result claim does not follow from a bounded source search.

7 What overlapping pair hulls miss

Exact pair support does not make intersected pair hulls the exact global hull. The failure already occurs on a path x - y - z in the unit cube, even if every square and edge product is present. Define

$$H_{xy} = \text{conv}\{(x, y, x^2, xy, y^2) : 0 \leq x, y \leq 1\}$$

and H_{yz} analogously. Let R intersect these two hulls while identifying the shared first and second moments of y . The full sparse quadratic hull is

$$H = \text{conv}\{(x, y, z, x^2, y^2, z^2, xy, yz) : 0 \leq x, y, z \leq 1\}.$$

Proposition 7.1 (A strict pair-hull gap). *The point*

$$\bar{v} = (1/2, 1/2, 4/5, 1/2, 5/16, 4/5, 3/8, 1/2) \quad (9)$$

belongs to R but not H . The quadratic function

$$D = (y - 1/4 - x/2)^2 + (y - 5z/8)^2 + x(1 - x) + z(1 - z) \quad (10)$$

has minimum $1/128$ on the cube, whereas its lifted minimum over R is zero.

Proof. The left pair measure assigns probability $1/2$ to each of $(x, y) = (0, 1/4), (1, 3/4)$. The right pair measure assigns probabilities $1/5, 4/5$ to $(y, z) = (0, 0), (5/8, 1)$. Both give $\mathbb{E}y = 1/2$ and $\mathbb{E}y^2 = 5/16$, and their remaining moments yield (9). Thus the point belongs to R .

In a global representing measure, $\mathbb{E}[x(1 - x)] = \mathbb{E}[z(1 - z)] = 0$ would force x, z to be binary almost surely. The given moments also force $\mathbb{E}(y - 1/4 - x/2)^2 = \mathbb{E}(y - 5z/8)^2 = 0$. The first identity requires $y \in \{1/4, 3/4\}$, and the second $y \in \{0, 5/8\}$, almost surely. Their supports are disjoint, so no global measure exists.

For fixed y , the square coefficients of x and z in D are $-3/4$ and $-39/64$. A minimum therefore has binary leaves. At those endpoints the last two terms vanish. Set $a = 1/4 + x/2$ and $b = 5z/8$. Then

$$(y - a)^2 + (y - b)^2 = 2(y - (a + b)/2)^2 + (a - b)^2/2.$$

The midpoint lies in $[0, 1]$. The smallest difference between $\{1/4, 3/4\}$ and $\{0, 5/8\}$ is $1/8$, giving $1/128$, attained at $(x, y, z) = (1, 11/16, 1)$. Each pair summand of D is nonnegative on its pair box, and both have zero expectation under the local witnesses. Their lifted minimum over R is therefore zero. $\square$

Expanding $D \geq 1/128$ gives a cut involving only the existing sparse quadratic coordinates. With m_u, s_u, c_{uv} denoting the first, square, and product coordinates, it is

$$2s_y - \frac{1}{2}m_y - c_{xy} - \frac{5}{4}c_{yz} + \frac{5}{4}m_x - \frac{3}{4}s_x + m_z - \frac{39}{64}s_z \geq -\frac{7}{128}. \quad (11)$$

It excludes (9) by $1/128$, with no xz product. The star oracle of Theorem 6.1 computes this support value exactly. Scaling the objective scales the gap; $1/128$ is an exact diagnostic value, not a universal limit on missing strength.

This comparison is against exact *pair* hulls, not against a fully coupled SDP relaxation. A dense moment matrix together with the missing nonedge product and its McCormick inequalities excludes this witness too. At the proposed moments, $\text{Var}(y) = 1/16$, $\text{Var}(x) = 1/4$, $\text{Var}(z) = 4/25$, and the two edge covariances saturate the positive Cauchy–Schwarz bounds. A global PSD moment matrix would therefore require $\text{Cov}(x, z) = 1/5$, hence $\mathbb{E}[xz] = 3/5 > \mathbb{E}[x] = 1/2$, contrary to $xz \leq x$. The star cut repairs this example without adding that nonedge coordinate; general dominance over SDP or RLT is not asserted.

There is a simple family behind this example. Suppose $a, a + b, c, c + d \in [\ell, u]$ and let $\delta = \min\{|s - t| : s \in \{a, a + b\}, t \in \{c, c + d\}\}$. For $0 \leq x, z \leq 1$, $\ell \leq y \leq u$,

$$(y - a - bx)^2 + (y - c - dz)^2 + b^2x(1 - x) + d^2z(1 - z) \geq \delta^2/2. \quad (12)$$

The left side is affine in each leaf after expansion, so its minimum occurs at binary leaves. The preceding midpoint calculation proves both validity and sharpness. Automatic selection of the four parameters is not supplied merely by this family; the star support algorithm is the implemented general mechanism within its stated class.

7.1 The stronger interface that makes tree gluing valid

The obstruction concerns inconsistent representing distributions, not an error in the individual hulls. Under a running-intersection tree of compact block domains, local probability measures glue if they have the *same complete marginal distribution* on every adjacent separator [13]. Root the tree; retain the root measure and attach each child’s new variables using its conditional distribution given the parent separator. Matching marginals preserve that child’s distribution, and running intersection prevents inconsistent assignments of previously introduced variables. Induction constructs a global representing measure. Conditional distributions exist for these compact Euclidean Borel spaces; finite-support gluing is ordinary conditional probability and finite summation.

For a binary scalar separator, its mean determines the complete distribution. For a separator known to lie in a fixed set of k distinct scalar values, matching moments of orders $0, \dots, k - 1$ determines atom probabilities through an invertible Vandermonde system. Neither observation applies to unrestricted continuous separators with only first and second moments. Means of several binary separator variables also do not determine their joint distribution.

The counterexample does not contradict exactness of the McCormick relaxation for a *pure bilinear* graph on a forest. On the unit box, an edge with means m_i, m_j and product w_{ij} has binary probabilities

$$p_{11} = w_{ij}, \quad p_{10} = m_i - w_{ij}, \quad p_{01} = m_j - w_{ij}, \quad p_{00} = 1 - m_i - m_j + w_{ij}.$$

The McCormick inequalities are their nonnegativity conditions. Adjacent edges share a binary marginal fixed by the common mean, so these measures glue along the forest. Prescribed square coordinates can forbid this binary choice, as they do in (9).

The companion [theory/overlap-review.md](#) also gives a nonlinear constrained example that survives separate block interval propagation and is repaired by one additional separator statistic. That example illustrates a different possible extension; it is not claimed to defeat global presolve or to be an automatic statistic-selection algorithm. This continuation implements the concrete alternative of selected larger star support cuts.

8 Implementation and model binding

8.1 What the certificate actually checks

The main entry points in [solver/certified.py](#) are `certify_support` and `replay_support`. Their features are SymPy expression objects with an explicit ordered symbol tuple; certificates never supply source code to be evaluated. The serialized format `support-cut-v1` binds the following data:

1. a typed expression tree over a fixed whitelist of operations;
2. the exact rational box and every supplied affine inequality;
3. the final coefficient vector through binary floating-point hexadecimal identities;
4. the selected arithmetic method and precision;
5. a complete dyadic subdivision proof, exact polygon candidate proof, or exact star partition proof;
6. the rational lower bound and downward-rounded binary right-hand side.

The polynomial path accepts rational algebra and nonnegative integer powers up to the stated degree cap. The general elementary path is univariate and accepts addition, multiplication, rational powers, exp, log, sin, cos, sinh, cosh, tanh, arctan, and absolute value, with π and e constants. Rational powers use the nonnegative-base real convention, with a strictly positive base for negative fractional exponents. An odd-root extension to negative arguments is not inferred. Negative integer powers require a denominator interval excluding zero. Elementary enclosures use the Arb library through python-flint [10].

For nonquadratic polynomials, the implemented Bernstein fallback is restricted to one or two variables and at most 1024 tensor coefficients. Higher-dimensional supported quadratics use the star oracle. The kernel degree cap and integration discovery caps are different: the general kernel admits integer powers up to 32, while the experiment’s discovery limits total degree to eight for one variable and four for a pair. These are explicit work limits, not mathematical barriers of the support construction.

Affine rows may exclude a subdivision cell only when an exact lower bound on the row’s left side exceeds its right side. If all cells are excluded, the record proves the supplied box-and-row domain empty. Otherwise each remaining cell needs a support lower bound. Replay reconstructs the dyadic children from the trusted parent box, rejects missing children, and recomputes the local enclosures or exact oracle. It then checks the final right-hand side against the minimum verified lower bound. Finite work limits return **incomplete**, with no cut; unsupported expressions return **unsupported**. A valid but insufficiently strong exact support value can likewise fail a requested separation target.

The trusted representation starts at the supplied expression objects. Simplifications performed before this boundary cannot be undone by the checker. The solver integration therefore also retains original parsed expression trees and native equalities. Loaded OSiL constants are binary64 values; the new cut proof treats those exact binary values as real constants. It is not a separate certificate that every original decimal token in an OSiL file has been modeled over exact decimal arithmetic. Saved file hashes and original-tree checks bind the experiment to that parsed model.

The import boundary is deliberately narrower than an ordinary numerical OSiL reader. Before simplification, a walk of every original source tree checks denominators, logarithm arguments, square roots, and fractional or negative powers on the complete declared box. An unproved domain restriction refuses the model in every mode. A valid model whose rows or nonlinear functions themselves narrow an overly broad declared box can therefore be refused. This admission check does not infer bounds from affine rows. Invalid sides, reversed bounds, NaN, and finite bounds or row sides reaching SCIP’s infinity sentinel are also refused.

PySCIPOpt can combine stored coefficients using binary64 arithmetic while assembling a row. The integration reconstructs the assembled expression’s stored coefficients exactly and compares its algebraic value with the exact source-tree value. It applies this whole-row check in *all four modes*, including the native baseline. A mismatch rejects the model import. Constant arithmetic subtrees are folded beforehand only when their exact value is representable as a finite binary64 number; variable-bearing nodes retain their native domain restrictions. This does not make all decimal-coefficient models admissible: ordinary coefficient folding can fail the exact test. Refusal is an applicability outcome and must remain in the denominator of the selected experiment population. The structured status **source_model_mismatch** carries its diagnostic and supplies no primal value, dual bound, or cut.

The same comparison checks candidate auxiliary definitions. After reformulation it checks entire rewritten rows with the auxiliary expressions substituted back. A failed auxiliary comparison declines the atom; a failed rewritten-row comparison retains the original row. The comparison establishes algebraic equality where the native source expressions are defined. Native domain enforcement remains necessary, and this is not a proof that all symbolic or later solver transformations preserve

every domain.

8.2 Discovery and native enforcement

`solver/integration.py` collects bounded nonlinear atoms with one or two original variables. It preserves additive terms and constant multipliers so that several functions of a shared variable remain visible. Atoms are identified by their original source trees; equal simplified formulas with different source domains are not silently merged. Original quadratic row terms are included. Pair candidates arise from a two-variable atom or an original affine row containing that pair. No exhaustive search over all variable pairs is needed.

Every selected atom receives an auxiliary $w = f(x)$ that is enforced by SCIP’s native nonlinear constraints. Rewritten objective and model rows use these auxiliaries. Original variables and auxiliary definitions remain present for nonlinear feasibility and branching. The same reformulation and aggregation restrictions are applied in the `control`, `all`, and `auto` modes. Thus `control` isolates the cost and effect of this representation from the added cuts. The `baseline` mode builds the parsed original model without these new auxiliaries.

The extension proposes selected quadratic stars with two to four leaves from their feature interaction graph. The exact support oracle also checks every retained affine row against the center structure; a candidate containing incompatible leaf–leaf rows is rejected, not silently relaxed while being called exact. Discovery caps the number of atoms at 256, blocks at 64, and coordinates per block at eight, including original coordinates and nonlinear features. Retaining a capped subset remains a valid outer relaxation; it is not an assertion that all dependencies in a large model were captured.

All added rows in this implementation are global root cuts. Their proof domains are original declared finite boxes together with the retained original affine rows. No numerical presolve bound or local node interval is silently promoted into a globally valid cut domain. The component adds neither custom feasibility enforcement nor custom branching or propagation. This deliberately keeps the cut’s trust scope smaller than the complete solver’s.

Before insertion, the component inspects the actual SCIP row’s columns, coefficients, constant, sides, and local flag. It requires exact agreement with the certified row after the explicit original-to-transformed variable mapping. A dropped variable, numerical fixing, aggregation substitution, changed coefficient, or changed effective right-hand side causes rejection. This conservative check supplies no proof for an unrecorded presolve deduction. The saved artifact includes the inspected row, so replay can check this binding rather than trust the generator’s claimed variable names alone.

8.3 Automatic activation and bounded repeated work

The direction LP separates the current lifted point from a finite sample hull after coordinate scaling. It has one free intercept and one coefficient in $[-1, 1]$ for each scaled coordinate. The samples only propose a direction. A one-variable grid has 65 points; a pair uses a 13-by-13 grid. Larger stars use 128 deterministic Halton points and box corners. Exact polygon vertices, edge midpoints, and a centroid, or exact star coordinate extrema, supply feasible seeds when affine equalities defeat a box grid. Candidate-row admission can use numerical tolerances, because a sample grants no authority to validate a cut. A screening witness, in contrast, must pass exact box-and-row feasibility.

The automatic policy requires shared univariate structure, a pair with a retained affine row, or a recognized larger star. It uses a larger minimum scaled violation ($5 \cdot 10^{-4}$ rather than 10^{-5}), stops trying a block after two failed useful certifications, and can reuse the checked convex-combination screen from Section 4. The screen is initialized only for source-polynomial features, whose domains

have no hidden elementary-function exclusions. A failed screen never causes a skip. Identical-direction and nearly unchanged query checks also avoid repeated work. These latter work-selection rules are heuristics: they may miss useful cuts and do not certify that more separation is unnecessary.

For exact quadratic support, a direction with insufficient violation still returns an attained minimizing parameter point. The integration can add that point to the numerical sample set and resolve the direction LP, with at most three exchange rounds. This addresses a concrete weakness of a fixed sample hull without assuming that a fixed number of exchanges makes separation complete. Samples used by the certified screen are reevaluated from their retained exact rational parameter values.

Within one model, cached samples, normalization, expression preparation, and feasible graph evaluations avoid repeated setup. Their domain is fixed. The public screening checker re-evaluates serialized graph samples; in-process cache reuse is not a reason to trust a foreign record.

The default limits are five root callbacks, 24 total cuts, six cuts per callback, 128 visited support cells, and subdivision depth 16. The total callback budget is the smaller of two seconds and 15% of the solve’s integration budget. Time checks occur between block operations, so this budget is cooperative and can be exceeded by a single expensive operation. The experiment additionally gives each worker an external hard process timeout. This distinction prevents a soft timing setting from being described as a strict worst-case bound.

The implementation records discovery, model construction, candidate generation, support certification, row insertion, and total callback time. Callback time is already included in SCIP solve time and must not be added a second time. Imports and worker startup are outside the in-process budget and are reported through outer process time.

8.4 The native sampling kernel

The optional C kernel in `solver/native_sampling.c` reuses integer powers while evaluating several polynomial features on the same sample points. NumPy remains the default. An explicit build attempt compiles a private temporary shared library; import and ordinary sampling do not start a compiler. Even after a build, the C path is selected only for at least 256 points and individual-variable degrees from three through 64. Small or low-degree cases remain with NumPy. Expression preparation has a bounded cache; values on mutable caller arrays are not cached.

This is a native *sampling* implementation, not a native SCIP nonlinear handler or a C implementation of exact support. Its outputs are floating point and can overflow. Candidate generation must reject nonfinite values; final validity still comes from the original exact features and the checked support bound. The microbenchmark and its unfavorable low-degree cases appear in Section 9.

9 Computational evidence and verification

9.1 The frozen comparison

The experimental question is whether the checked block component is useful after accounting for discovery, reformulation, sampling, certification, and repeated work. The four modes are the original parsed model (**baseline**), the common auxiliary formulation without new cuts (**control**), every supported block within the same work caps (**all**), and the automatic policy (**auto**). Native SCIP separation remains enabled.

The saved protocol includes 13 predefined mechanism and control models and a held-out slice of 24 MINLPLib models. The latter excludes the 123 names occurring in two specified earlier repository campaigns, filters by model metadata and parser support, and selects by a fixed SHA256

ordering from 279 eligible models. It does not filter by new handler applicability or observed outcomes. This is held out from the two named campaigns, not from every possible past use of public MINLPLib instances. The initial six-model proposal was expanded to 24 before new optimization outcomes, keeping the same order. The archived metadata includes 11 held-out models with integer variables and seven classified as convex. Four previously studied instances—`btest14`, `waterno2_06`, `ghg_2veh`, and `chp_partload`—form a separate diagnostic suite and are excluded from the held-out summary.

Each main run has a six-second soft in-process budget, one SCIP thread, BLAS/OpenMP thread environment variables set to one, seed zero, and relative gap tolerance 10^{-4} . Each independent worker has a hard 20-second process timeout. A separate one-node comparison uses two-second budgets on the 24 held-out models and five predefined mechanism cases. Imports, process startup, and independent post-run checking are measured separately. No full run is warm-started from the one-node comparison. The host also runs unrelated research tasks, so subsecond timing differences are descriptive.

Discovery, model construction, and individual support calls are nonpreemptive, so the soft budget can be exceeded; observed overshoots are retained. If the hard timeout kills a process before its result is written, its cut count is unknown. An absent record is not evidence of zero cuts or successful replay. Strict source/native import refusals are reported separately from optimization failures. These are refusals by this research importer, shared by all modes, rather than native SCIP failures. Selected-population totals retain their denominator, but admitted counts and refusal counts must be shown separately; only admitted models measure solver behavior.

Caching, native sampling, and selected star merging have separate ablations when applicable. Repeats use a fixed second-seed subset. The `no_cache` switch is a combined reuse-policy ablation: it disables exact-support sample exchange, previous-query skipping, and certified mixture screening as well as sampled-value reuse. Its comparison cannot isolate the causal effect of caching alone. The aggregate campaign budget is bounded. Results preserve errors, timeouts, domains rejected by the new component, and unfavorable outcomes. Any rerun after an identified defect retains the original record and explains the implementation change.

An independent evaluator checks every returned original variable vector against original bounds, integrality, expression domains, rows, and objective using scaled tolerance 10^{-5} . This is a numerical residual check. The checker also compares dual bounds to known synthetic optima or archived feasible objective values with explicit numerical tolerances. Neither check certifies a global dual bound. Fresh-process support replay instead checks each saved cut against the reconstructed model binding; a missing or failed replay is not counted as a certified row.

9.2 Primary outcome: fewer held-out solves with cuts

The completed campaign contains 316 runs and took 443.6 seconds of supervised wall time. Its retained [campaign-v1/summary.json](#), [campaign-v1/records.jsonl](#), source snapshot, model files, and per-worker logs preserve every outcome. The primary comparison is Table 1. “Solved” means SCIP reported `optimal` or `gaplimit`, returned a finite primal objective, and passed the independent original-model numerical check without a reference-bound conflict. It is a numerical stopping outcome, not an exact optimality certificate.

All four modes refused the same three held-out models because their logarithm domains were not proved on the declared boxes: `syn15m`, `cvxnonsep_psig30r`, and `syn10hfsg`. The inherited builder also could not handle the genuine variable-exponent terms of `cvxnonsep_pcon40r`. These four cases were never comparable solver attempts. They remain in the selected count of 24; the solver comparison concerns 20 admitted cases. This reports the limitations of the research importer,

Suite	Mode	Selected	Admitted	Solved	Cuts	Time (s)
Synthetic	baseline	13	13	12	0	6.57
Synthetic	control	13	13	13	0	1.66
Synthetic	all	13	13	13	89	2.51
Synthetic	auto	13	13	13	72	2.45
Held out	baseline	24	20	19	0	17.40
Held out	control	24	20	19	0	20.66
Held out	all	24	20	18	230	25.41
Held out	auto	24	20	18	106	23.01

Table 1: Primary six-second runs. Counts retain the selected population and show actual admission separately. Cuts are recorded rows. Time is the sum of available in-process integration and source-reading times, excluding worker startup and post-run checking.

not the capabilities of native SCIP on the refused models.

The additional held-out timeout in both cut modes was `genpooling_lee2`: baseline and control solved it in approximately 5.61 and 4.89 seconds, while both cut modes reached the six-second limit. `kall_circles_c6b` remained time-limited in every mode. Thus the observed primary outcome is adverse for enabling the new separator by default. Automatic selection reduces work relative to unconditional activation but does not recover the lost solve.

The synthetic improvement in solved count also needs its control: `quartic_balance_8` is unsolved by baseline within six seconds, but control already solves it without new cuts. Both cut modes solve it too. The count change from 12 to 13 therefore cannot be attributed to the new rows. On the exact overlap instance, native baseline already reaches the optimum numerically in about 0.10 seconds; the merged cut modes take about 0.50 and 0.48 seconds. The strict mathematical pair-hull gap remains useful, but this full solver already has other ways to address that instance.

In the separate historical suite, only `waterno2_06` passes the common importer, and every mode is time-limited. Its numerical lower bounds are approximately 26.59, 21.17, 7.05, and 2.41 for baseline, control, all, and auto. The other three historical instances are refused before solving. This diagnostic does not extend the earlier historical gains to the new verified component.

9.3 Root strength, work selection, and ablations

Held-out one-node comparisons are mixed. Relative to baseline, `all` has three better bounds, 15 ties, and two worse bounds; `auto` has three better, 14 ties, and three worse. Four comparisons are unavailable because of the common import limitations. The comparison tolerance is 10^{-6} times the larger bound scale. These are the numerical bounds at the end of the separate two-second, one-node runs, including native separation, rather than exact values of the new block relaxations.

On the primary held-out runs, automatic selection reduces candidate LP calls from 1073 to 323, recorded cuts from 230 to 106, and total callback time from 5.59 to 3.05 seconds relative to `all`. Support certification accounts for 0.70 and 0.34 seconds respectively. There are 95 and 14 unsuccessful support attempts; these include insufficient violation, unsupported structure, and exhausted budgets, not invalid accepted cuts. Across all phases, the automatic convex-combination screen is queried 506 times and permits two skips. Its mathematical guarantee is tested, but this campaign does not demonstrate large savings from the screen itself.

The recorded full held-out in-process totals are 17.40, 20.66, 25.41, and 23.01 seconds. The corresponding sums of outer worker time, including startup and checking, are 36.66, 40.12, 45.44, and 42.49 seconds. These totals retain available refusal costs; an unsupported worker without an

in-process result has only its outer time. The shorter automatic callback total does not establish a broad complete-solve gain. One-seed, shared-host timing and short budgets also preclude precise estimates of small relative differences.

Disabling star merging on the overlap instance changes the all/auto times to about 0.19/0.10 seconds, with numerical solution status unchanged. The larger exact support class therefore has a strict mathematical diagnostic but no demonstrated runtime advantage in this native-solver case. The combined reuse-policy ablation also keeps the selected mechanism cases numerically solved and shows small mixed time differences. Because it removes exchange, screening, and repeated-query checks along with caches, it cannot isolate a cache-only effect. The fixed second-seed runs are retained in the summary and do not constitute a population-level statistical study. No algorithm tuning was performed in response to these campaign outcomes.

9.4 Replay coverage and numerical residual checks

Fresh-process replay in [campaign-v1/replay.json](#) checks all 1082 recorded cuts against original-model feature bindings, complete proof domains, final coefficients, and saved actual native rows. All 1082 pass. The replay binds 308 saved original-model outputs, verifies 101 frozen source/input hashes, and rejects all 12 coefficient, right-hand-side, domain, variable, scope, and native-row tampering controls.

The eight remaining jobs are full and one-node attempts on `cvxnonsep_pcon40r` in four modes. They fail on the unsupported variable exponent before optimization according to the worker trace, but lack original-model and cut logs. Their logged cut count stays unknown; they are not represented as successfully replayed runs with zero cuts. There are 36 structured importer refusals across all phases, retained with their model records. No hard process timeout or aggregate-budget exhaustion occurs. Soft-budget overshoots remain in the raw summary; they are not converted into extra allotted time.

The independent evaluator checks 270 returned original-variable incumbents; all pass its scaled 10^{-5} test. No checked root or final dual bound conflicts with its analytic optimum or archived feasible reference under the stated tolerance. These results are numerical sanity checks and do not convert a SCIP bound, its incumbent, or its search into an exact proof. The independently derived metrics audit and full integration review are retained alongside the replay output.

9.5 The exact overlap mechanism

The standalone [experiments/star-mechanism.json](#) stores the two rational pair measures, their shared moments, the three-piece exact center partition, and the attained star minimizer for Proposition 7.1. The pair-hull objective is exactly zero and the assembled support value is exactly $1/128$, attained at $(1, 11/16, 1)$. The replay accepted this proof and rejected a changed objective constant. This is an exact algebraic and implementation diagnostic, independent of SCIP and of its root-bound heuristics. Its source hashes are retained in the same record. It establishes the specific missing strength and the oracle’s recovery of it; it does not establish a runtime advantage over native dense SDP/RLT treatment.

9.6 Native sampling measurements

The retained [implementation/native-kernel-benchmark.json](#) contains 35 feature-count/grid-size combinations, source hashes, versions, preparation times, and the chosen backend. Table 2 gives selected 4096-point results. The timings are medians of 11 batches of 20 warmed evaluations. Build and load took 0.268 seconds; expression preparation is measured separately.

Features	NumPy (μ s)	C (μ s)	NumPy/C
x^2	5.20	30.20	0.17
x, y, xy, x^2, y^2	30.81	48.46	0.64
x^4	236.94	27.76	8.54
$x, \dots, x^6$	972.95	43.51	22.36
24 shifted quartics	6176.34	390.89	15.80

Table 2: Sampling kernels at 4096 points; these are not complete-solve speedups. The dispatch retains NumPy for the low-degree rows.

The 24 quartics are $(x + iy/8)^4 + ixy/13$, $i = 1, \dots, 24$. At 32 points the single-quartic C call was slower than NumPy, and the low-degree cases remain slower in C even at 4096 points. The selected dispatch and explicit build requirement preserve these unfavorable findings. In a main separation grid of 65 or 169 points, the default native threshold is not reached. Thus these microbenchmarks alone cannot explain a solver improvement in that experiment. The main campaign does not explicitly build the C sampler and uses NumPy. Explicit compilation occurs before the integration function’s soft budget; a wrapper must count it in outer process time when assessing a compiled-sampler run.

9.7 Targeted checks and actual-file review

The combined targeted command recorded in [VERIFICATION.md](#) passed 186 tests and 53 subtests across the new solver, theory, independent-review, and experiment-harness files. A later pre-freeze nonfinite-objective regression was checked by a focused four-test harness rerun; three of those tests were already in the combined run. These numbers are separate executed checks, not a claim that one final command ran 187 distinct tests. The archived command avoids the frozen source copies and collects no unrelated repository tests.

The kernel tests exercise interior minima, exact final coefficient rounding, complete subdivision coverage, strict row exclusions, domain singularities, empty domains, and tampered model or proof records. Screening tests exercise exact mixture normalization, normalization-specific violation bounds, interval graph values, infeasible samples, and replay. The polygon and star tests compare independently formulated special cases, degeneracies, and the exact overlap example. An additional star audit enumerates stationary points on full-space faces rather than repeating the leaf algorithm. Native sampling tests cover reference values, alignment, empty arrays, unsupported data, and compiler or storage failure.

The full-space star audit compared 155 cases (76 feasible and 79 empty), checked 298 returned pieces, and rejected seven mutations. A second audit used independent projection and verified leaf rules over 786 complete center intervals in 88 cases, making 7063 exact alternative-rule comparisons and rejecting three mutations. The two checks exercise different contracts: attained global optima and validity of every reported conditional piece.

The numerical and mathematical reviews read actual source and certificate files. Their independence means a different research agent checked the saved artifacts; it does not mean a separately implemented arithmetic library, a formal proof assistant, or journal peer review. The companion claim map records the precise test and review paths. Verification is restricted to this topic; no project-wide test run or CI inspection is claimed.

10 Relationship to established results

The local source audit is [literature/README.md](#), with retained primary documents, a source manifest, and focused comparisons for composition and stars. It is a targeted audit, not a proof of novelty. In particular, the following ingredients should not be presented as discoveries of this continuation.

Simultaneous support separation is established. Liers et al. [15] attribute the linear-combination hull characterization to Ballerstein’s dissertation [2] and develop simultaneous separation for gas networks. Tawarmalani’s inclusion certificates [21] already explain why compatible representing measures matter when combining hulls. The term “inclusion certificate” there refers to a mathematical measure representation, not this implementation’s serialized cut artifact. The dissertation download was unavailable during this continuation; detailed historical claims about its Chapter 5 remain attributed to the earlier repository audit, while the hull characterization is corroborated by Liers et al.’s inspected primary text.

Vector composite relaxations are also established. He and Tawarmalani [7] transfer simultaneous separation through an oracle-based construction; [8] gives common triangulation conditions for combining simultaneous hypographs; [9] treats vector composite graphs and shared discretization. The checked 2024 preprint is its third version, whose Proposition 28 and Corollary 29 explicitly address vector graphs. Zhu, He, and Tawarmalani [22] develop axis-aligned simultaneous relaxations and convergence on coupled domains, with a large MINLP Lib root-bound study. Li et al. [14] construct univariate polynomial envelopes through continuous Graham-scan ideas under their stated numerical model. The present cut contract therefore concerns a particular supported representation, exact final coefficients, and measured implementation behavior, rather than the introduction of shared-variable convexification.

Bernstein enclosure and rigorous final affine shifts predate this implementation. Garloff and Smith [6], Section 5, explicitly choose an affine candidate numerically and validate a final shift with interval Bernstein coefficients. Garloff, Jansson, and Smith [5] study inclusion under domain shrinkage. Polynomial dependencies and selective products also appear in reformulation-linearization methods [20]. The new record binds an explicit expression graph, complete domain, and final row so that the concrete claim can be replayed. This is an implementation contract to evaluate, not a new general principle of validated numerics.

Exact low-dimensional quadratic hull descriptions are known from Anstreicher and Burer [1]. Their Theorem 7 uses triangulation for polytopes of dimension at most three, and their low-order completely positive representation explains why these classes have exact semidefinite descriptions. The stationary-candidate oracle of Theorem 5.1 is a direct, exact arithmetic way to compute support in two dimensions. It does not supersede or introduce that hull theory. Locatelli [17] and Müller, Serrano, and Gleixner [18] already use constrained two-dimensional geometry for stronger bilinear treatment. Our vector of quadratic features is broader than one product, but improvement over independent McCormick inequalities can duplicate strength already available in native solver cuts.

For stars, Del Pia and Khajavirad [19] directly cover box-only support through their forest algorithm. Classical parametric QP [3] explains positive-curvature affine responses. Khajavirad [11] provides tight sparse SDP relaxations under additional sign and graph hypotheses. Locatelli’s simplex-envelope stars [16] refer to a different convexity graph and domain. The explicit all-sign center–leaf row extension here must be judged in this context; an unmatched exact theorem in a short search does not establish priority.

The overlap example illustrates a limitation of a chosen finite interface. It does not refute sparse moment convergence or known finite-extraction results [12, 13]. Full separator distributions, or enough moments to identify them under additional structure, satisfy a stronger gluing condition

than equal means and squares. Neither the bounded cut loop nor the moment hierarchy is shown to dominate the other generally.

Finally, automatic recognition is already a solver capability. SCIP’s nonlinear handlers, projected bilinear envelopes, RLT cuts, and selected moment minors are described in [4]. The native baseline therefore remains the appropriate primary comparator. The contribution supported by this report is the integrated checked-row implementation, explicit support classes and counterexamples, and its transparent positive and negative evidence. Broad performance or first-method claims require evidence beyond this bounded continuation.

11 What is complete and what remains outside the result

The supported program joins a complete domain-bound cut contract to automatic generation, exact coupled quadratic support, a larger structured support class, and saved comparative evidence. None of the proofs asserts tractable exact separation for arbitrary overlapping nonlinear blocks. Acyclic overlap alone does not make first-moment agreement sufficient. General-function domain certification can reject expressions near singularities, and bounded polynomial subdivision can return an inconclusive result. These behaviors preserve soundness at the cost of applicability or strength.

The computational standard is correspondingly limited. Fewer nodes or a better root relaxation on a mechanism case is useful evidence for the mechanism. It does not establish a population-wide runtime improvement. The native sampling kernel speeds only candidate generation; validated support and model checking retain their costs. The experiments retain reformulation controls, unsuccessful activations, rejected domains, timeouts, and numerical feasibility failures so that these costs are visible.

The held-out solved count decreases: both cut modes solve 18 of the 20 admitted cases, versus 19 for baseline and control. The 24-case selection also exposes three strict domain refusals and one unsupported variable-exponent model, all common to the four modes. The synthetic solved-count gain is already obtained by the reformulation control. The recommended operational status is therefore an optional research component for supported structures, with default activation unsupported by this evidence. The exact support classes, model-bound certificates, and obstruction examples are completed results despite this adverse performance finding.

The full solver remains a numerical optimizer with native model enforcement. Extending the cut records to certify all original-model transformations, presolve deductions, LP bounds, spatial branching, and primal feasibility is a different end-to-end certification task. The present result supplies checked rows that could enter such a system.

References

- [1] Kurt M. Anstreicher and Samuel Burer. Computable representations for convex hulls of low-dimensional quadratic forms. *Mathematical Programming*, 124:33–43, 2010. Theorem locators in this report refer to the February 6, 2007 author preprint.
- [2] Martin Ballerstein. *Convex Relaxations for Mixed-Integer Nonlinear Programs*. PhD thesis, ETH Zurich, 2013. Chapter 5 details retained from the earlier repository audit; full PDF retrieval returned HTTP 429 in this continuation.
- [3] Alberto Bemporad, Manfred Morari, Vivek Dua, and Efstratios N. Pistikopoulos. The explicit linear quadratic regulator for constrained systems. *Automatica*, 38(1):3–20, 2002.

- [4] Ksenia Bestuzheva, Antonia Chmiela, Benjamin Müller, Felipe Serrano, Stefan Vigerske, and Fabian Wegscheider. Global optimization of mixed-integer nonlinear programs with SCIP 8. *Journal of Global Optimization*, 91(2):287–310, 2025. Online publication 2023; detailed section locators refer to arXiv:2301.00587v1, the longer preprint.
- [5] Jürgen Garloff, Christian Jansson, and Andrew Paul Smith. Inclusion isotonicity of convex-concave extensions for polynomials based on Bernstein expansion. *Computing*, 70(2):111–119, 2003.
- [6] Jürgen Garloff and Andrew P. Smith. Rigorous affine lower bound functions for multivariate polynomials and their use in global optimisation. In *Proceedings of the 1st International Conference on Applied Operational Research*, volume 1 of *Lecture Notes in Management Science*, pages 199–211, 2008. Also issued as Konstanzer Schriften in Mathematik und Informatik 250, June 2008.
- [7] Taotao He and Mohit Tawarmalani. A new framework to relax composite functions in nonlinear programs. *Mathematical Programming*, 190:427–466, 2021.
- [8] Taotao He and Mohit Tawarmalani. Tractable relaxations of composite functions. *Mathematics of Operations Research*, 47(2):1110–1140, 2022.
- [9] Taotao He and Mohit Tawarmalani. MIP relaxations in factorable programming. *SIAM Journal on Optimization*, 34(3):2856–2882, 2024.
- [10] Fredrik Johansson. Arb: Efficient arbitrary-precision midpoint-radius interval arithmetic. *IEEE Transactions on Computers*, 66(8):1281–1292, 2017.
- [11] Aida Khajavirad. Tight semidefinite programming relaxations for sparse box-constrained quadratic programs, 2026. Version 2.
- [12] Jean B. Lasserre. Global optimization with polynomials and the problem of moments. *SIAM Journal on Optimization*, 11(3):796–817, 2001.
- [13] Jean B. Lasserre. Convergent SDP-relaxations in polynomial optimization with sparsity. *SIAM Journal on Optimization*, 17(3):822–843, 2006.
- [14] Tianwei Li, Daniel Ovalle, Barnabas Póczos, Carl Laird, Ignacio Grossmann, and Javier Pena. Efficient convexification of Kolmogorov-Arnold networks with polynomial functional forms via a continuous Graham scan approach, 2026. Version 1 inspected.
- [15] Frauke Liers, Alexander Martin, Maximilian Merkert, Nick Mertens, and Dennis Michaels. Solving mixed-integer nonlinear optimization problems using simultaneous convexification: a case study for gas networks. *Journal of Global Optimization*, 80(2):307–340, 2021.
- [16] Marco Locatelli. Convex envelopes of some quadratic functions over the n-dimensional unit simplex. *SIAM Journal on Optimization*, 25(1):589–621, 2015.
- [17] Marco Locatelli. Convex envelopes of bivariate functions through the solution of KKT systems. *Journal of Global Optimization*, 72(2):277–303, 2018.
- [18] Benjamin Müller, Felipe Serrano, and Ambros Gleixner. Using two-dimensional projections for stronger separation and propagation of bilinear terms. *SIAM Journal on Optimization*, 30(2):1339–1365, 2020.

- [19] Alberto Del Pia and Aida Khajavirad. Treewidth and the complexity of box-constrained quadratic programs, 2026. Version 1, September 28, 2026.
- [20] Hanif D. Sherali and Cihan H. Tuncbilek. New reformulation linearization/convexification relaxations for univariate and multivariate polynomial programming problems. *Operations Research Letters*, 21(1):1–9, 1997.
- [21] Mohit Tawarmalani. Inclusion certificates and simultaneous convexification of functions. Optimization Online manuscript, 2010.
- [22] Haisheng Zhu, Taotao He, and Mohit Tawarmalani. Axis-aligned relaxations for mixed-integer nonlinear programming, 2026. Version 1 inspected.